3.b Lambdas

Higher-Order Functions & Lambda Expressions in Kotlin

Contents

1 **Higher-Order Functions**
Functions that take functions as parameters or return a function

2 **Function Types**
Notation, nullable types, type aliases & instantiation

3 **Lambda Expression Syntax**
Full syntax form, parameter types, return values & type inference

4 **Trailing Lambdas & the it Parameter**
Passing lambdas outside parentheses & implicit single parameter

5 **Closures & Anonymous Functions**
Accessing outer scope variables & functions without a name

6 **Function Literals with Receiver**
Lambdas with receiver objects & type-safe builders

Higher-Order Functions

A higher-order function is a function that takes functions as parameters or returns a function.

Example: fold as a higher-order function

```
fun <T, R> Collection<T>.fold(
    initial: R,
    combine: (acc: R, elem: T) -> R // function type parameter
): R {
    var accumulator = initial
    for (element in this) {
        accumulator = combine(accumulator, element) // uso da função
    }
    return accumulator
}
```

```
val items = listOf(1, 2, 3, 4, 5)
println(items)

val product = items.fold(1, Int::times)
println(product)

val result = 5.times(3)
println("5 times 3 = $result")
```

```
Output:
[1, 2, 3, 4, 5]
120
```

```
Método da classe Int (e das outras classes de tipos):
public final operator fun times(other: Int): Int
```

Key Concepts

1. Kotlin functions are first-class: they can be stored in variables and data structures
2. Can be passed as arguments to and returned from other functions
3. Lambda expressions and function references are function literals — passed as expressions

```
data class Product(val name: String, val price: Double)

val products = listOf(
    Product("Book", 15.99),
    Product("Pen", 2.50)
)

val total = products.fold(0.0) {
                        acc, product -> acc + product.price }
println("Total: %.2f".format(total))
```

```
Output:
Total: 18,49
```

Function Types

Notation

```
(A, B) -> C     // Takes A and B, returns C
() -> Unit      // No params, returns nothing
A.(B) -> C      // Function type with receiver A
```

Nullable Types

Wrap in parentheses:

```
((Int, Int) -> Int)?
-- This is a nullable function definition
-- var operation: ((Int, Int) -> Int)? = null
```

Combining Types

```
(Int) -> ((Int) -> Unit)
-- Arrow is right-associative
-- A function that returns another function
```

Type Aliases

```
typealias Predicate<T> = (T) -> Boolean
-- Creates an alias for a function type
-- val isEven: Predicate<Int> = { it % 2 == 0 }
-- println(isEven(4))  // true
```

Named Parameters

```
(x: Int, y: Int) -> Point
-- For documenting meaning
```

```
val addAndPrint: (Int) -> ((Int) -> Unit) = { a -> { b -> println(a + b) } }
val f = addAndPrint(3) // 3 is a
f(4)   // 4 is b - Output: 7
addAndPrint(3)(4)
```

Lambda Expression Syntax

Full Syntactic Form (example with sum function)

```
val sum: (Int, Int) -> Int = { x: Int, y: Int -> x + y }
```

{ parameters –> body }
Function body always surrounded by **{ }**
(Int, Int) –> Int is the function definition
Functions as types – can be used as any type in variables, arguments, return types, …

Simplified Forms

Type inference

```
val sum = { x: Int, y: Int -> x + y }
```

Omit optional annotations

```
val sum = { x, y -> x + y }
```

```
val sum: (Int, Int) -> Int = { x: Int, y: Int -> x + y }
println(sum(1, 2))

val sum2: (Int, Int) -> Int = { x, y -> x + y }
println(sum2(1, 3))

val sum3 = { x: Int, y: Int -> x + y }
println(sum3(1, 4))

var f = sum3   // type is inferred as (Int, Int) -> Int

println(f(1, 5))

val sub3 = { x: Int, y: Int -> x - y }
f = sub3
println(f(1, 6))
```

```
Output:
3
4
5
6
-5
```

Return Value

The last expression in a lambda is implicitly the return value.
If the expected return type is Unit, the value of the last expression is ignored.

Trailing Lambdas & the it Parameter

Trailing Lambda Syntax

If the last parameter of a function is a function, the lambda can be placed outside the parentheses.

```
// Lambda as last argument:
items.fold(0, { acc, e -> acc + e} )
items.fold(0) { acc, e -> acc + e} // outside parent.
```

```
// Lambda is the only argument:
fun run5(block: () -> Unit) { block() }
run({ println("Hello") })
run { println("Hello") } // outside parenthesis
```

```
val map = mapOf(
    "Alice" to 18,
    "Bob" to 15,
    "Charlie" to 20
)

map.forEach({ (_, value) -> println(value) })
map.forEach { (_, value) -> println(value) } // o. p.
```

The it Implicit Parameter

If a lambda has only one parameter and the compiler can infer the type, the parameter is implicitly named **it**.

```
ints.filter { it > 0 }
// Same as: ints.filter { x -> x > 0 }
```

```
val names = listOf("Alice", "Bob", "Charlie")
names
  .filter { it.length > 3 } // Trailing lambda + it
  .map { it.uppercase() }   // Chained trailing lambdas
  .forEach { println(it) }  // Output: ALICE, CHARLIE
```

Explicit returns in lambdas: return@label

```
ints.filter {
    val shouldFilter = it > 0
    return@filter shouldFilter
}
```

Closures & Anonymous Functions

Closures

A lambda can access its closure — the variables declared in the outer scope. Variables captured in the closure can also be modified:

```
var sum = 0
ints.filter { it > 0 }
    .forEach { sum += it }
print(sum) // sum is modified!
```

Unlike Java, Kotlin lambdas can modify captured variables (not limited to final/effectively final).

Anonymous Functions

Like a regular function but without a name. Allows specifying the return type explicitly:

Expression body:

```
fun(x: Int, y: Int): Int = x + y
```

Block body:

```
fun(x: Int, y: Int): Int {
    return x + y
}
```

Key Difference: return Behavior

Lambda: return exits the enclosing function (non-local return). Use return@label for local return.

Anonymous function: return exits the anonymous function itself (local return), like a regular fun.

Local and Non-Local Returns in Lambdas

Non-Local Return (in lambdas)

A bare return inside a lambda exits the enclosing function, not just the lambda. This is called a non-local return.

```
fun findFirst(numbers: List<Int>) {
    numbers.forEach {
        if (it == 3) return
        println("$it")
    }
    println("Done")
}
val numbers = listOf(1, 2, 3, 4, 5)
findFirst(numbers)
// Prints: 1 2 (exits findFirst! Non-local return)
```

Local Return (using labels)

Use return@label to return only from the lambda (local return). The enclosing function continues executing.

```
fun printAll(numbers: List<Int>) {
    numbers.forEach {
        if (it == 3) return@forEach
        println("$it")
    }
    println("Done")
}
// … Prints: 1 2 4 5 Done (local return)
```

Anonymous Function: Always Local Return

In an anonymous function, return always exits the anonymous function itself — never the enclosing function. Use anonymous functions when you need guaranteed local return behavior.

```
numbers.forEach(fun(value: Int) {
    if (value == 3) return
    println("$value")
})
// … Prints: 1 2 4 5 (local return)
```

Key Rule: Non-local returns are only allowed in lambdas passed to **`inline`** functions (like forEach, map, etc.).

Function Literals with Receiver

Function types with receiver, such as A.(B) -> C, can be instantiated with a special form of function literals. Inside the body, the receiver object becomes an implicit **this**.

Function Type with Receiver

```
val sum: Int.(Int) -> Int = { other ->
    this + other
}
println(1.sum(2)) // prints 3
```

Like Extension Functions

Inside the function literal, the receiver object passed to a call becomes an implicit **this**. You can access members of the receiver without additional qualifiers.

Type-Safe Builders

```
fun buildString(
    action: StringBuilder.() -> Unit
): String {
    val sb = StringBuilder()
    sb.action() // call lambda with receiver
    return sb.toString()
}
```

Usage:

```
val result = buildString {
    append("Hello, ")
    append("World!")
}
```

Key pattern for Kotlin DSLs and builders!

With and without Receiver

Function Type with Receiver

```
fun buildString(
    action: StringBuilder.() -> Unit
): String {
    val sb = StringBuilder()
    sb.action() // call lambda with receiver
    return sb.toString()
}

val result2 = buildString {
    append("Hello, ")
    append("World!")
}
println(result2)  // Output: Hello, World!
```

Function Type without Receiver

```
fun buildStringNR(
    action: (StringBuilder) -> Unit
): String {
    val sb = StringBuilder()
    action(sb)
    return sb.toString()
}

val result3 = buildStringNR { sb ->
    sb.append("Hello, ")
    sb.append("World!")
}
println(result3) // Output: Hello, World!

val result4 = buildStringNR {
    it.append("Hello, ")
    it.append("World!")
}
println(result4) // Output: Hello, World!
```

Practice Exercises

Exercise 1: Higher-Order Function

Write a higher-order function applyOperation that takes two Ints and a function (Int, Int) -> Int. Call it with a lambda that multiplies the two numbers.

Solution

```
fun applyOperation(
    x: Int,
    y: Int,
    operation: (Int, Int) -> Int
): Int {
    return operation(x, y)
}

fun main() {
    val result = applyOperation(3, 4) { a, b -> a * b }
    println(result)
}
```

```
Output:
12
```

Concepts Used:

- Higher-order function
- Function type (Int, Int) -> Int
- Trailing lambda syntax
- Lambda expression { a, b -> ... }
- Passing function as last argument

Exercise 2: Closure & the it Parameter

Use a lambda with a closure to count how many positive numbers are in a list. Use filter with the it implicit parameter and modify a variable from the outer scope.

Solution

```
fun main() {
    val numbers = listOf(-3, -1, 0, 2, 5, 7)

    // Using filter with 'it'
    val positives = numbers.filter { it > 0 }

    // Closure: modifying outer variable
    var count = 0
    positives.forEach { count++ }

    println("Positives: $positives")
    println("Count: $count")
}
```

```
Output:
Positives: [2, 5, 7]
Count: 3
```

Concepts Used:

- Trailing lambda with filter
- Implicit it parameter
- Closure (capturing outer var)
- Modifying captured variable
- String templates with $

Exercise 3: Function Literal with Receiver

Implement a function **buildCsv** using a **lambda with receiver** that allows building CSV content in a DSL style.

Solution

```
class RowBuilder {
    private val fields = mutableListOf<String>()
    fun field(value: String) { fields.add(value) }
    fun build(): String = fields.joinToString(",")
}

class CsvBuilder {
    private val sb = StringBuilder()

    fun row(action: RowBuilder.() -> Unit) {
        val row = RowBuilder(); row.action()
        sb.append(row.build());  sb.append("\n")
    }

    fun build(): String = sb.toString()
}


fun buildCsv(action: CsvBuilder.() -> Unit): String {
    val builder = CsvBuilder(); builder.action();
    return builder.build()
}
```

Concepts Used:

- Function type with receiver
- .() -> Unit notation
- Lambda with implicit receiver (this)
- Trailing lambda syntax
- Type-safe builder pattern

```
val csv = buildCsv {
    row { field("Alice"); field("24"); field("Portugal") }
    row { field("Bob"); field("31"); field("Spain") }
}

println(csv)
```

```
Output:
Alice,24,Portugal
Bob,31,Spain
```

Your Turn — Practice Exercises

1 Lambda Refactor

Rewrite this function call using a trailing lambda instead of passing the lambda as a regular argument: items.fold(0, { acc, e -> acc + e })

Hint: Move the lambda outside the parentheses.

2 Type Inference Challenge

Write a lambda assigned to a variable without specifying the variable's type. Then write one where you must specify the type. What's the difference?

Hint: Think about when the compiler can infer types.

3 Anonymous vs Lambda

Write both a lambda expression and an anonymous function that filter a list of strings to those with length > 5. How does return behave differently?

Hint: return in lambda = non-local return.

4 Underscore Parameters

Given a map, use forEach with a destructured lambda to print only values. Use _ for the unused key parameter.

Hint: map.forEach { (_, value) -> ... }

Challenge Exercises

5 Custom DSL Builder

Create a mini-HTML builder DSL using function literals with receiver. Implement html { }, body { }, and p { } functions that nest and produce a formatted string.

Hint: Each builder class needs a method that takes a lambda with receiver.

6 Pipeline Composition

Write a compose function that takes two lambdas (Int) -> Int and returns a new lambda that applies both in sequence. Use it to chain doubling and adding-one operations.

Hint: A function can return a lambda that captures its arguments.

Tips for Success

- Experiment in the Kotlin Playground
- Start simple, then add complexity
- Check return types with the IDE
- Test lambda vs anonymous function returns
- Read the official Kotlin lambdas docs
- Practice with collection operations daily